DevOps & Tools · concept board

Git & GitHub

Commits, branches, merge vs rebase, undoing, team workflow

1. How Git thinks

3. Undoing things

2. Merge vs rebase

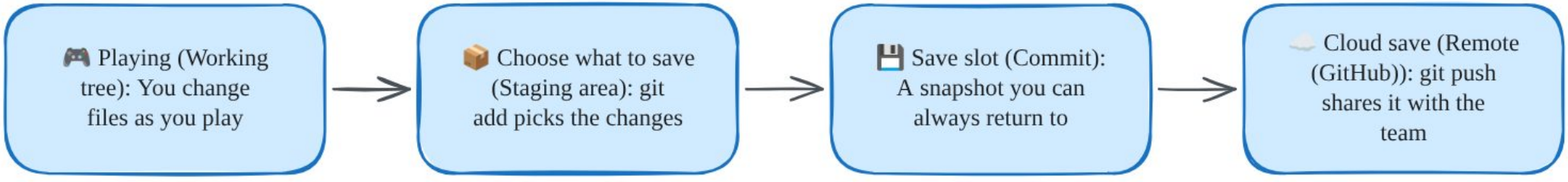
4. Team workflow on GitHub

Draw on any slide, then Present (arrow keys) or export the board as a PDF.

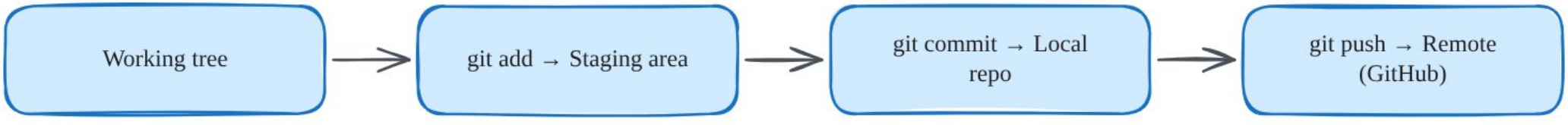
1. How Git thinks

Definition

Git is a distributed version control system. A commit is a snapshot of the project + a pointer to its parent(s); a branch is just a movable pointer to a commit.



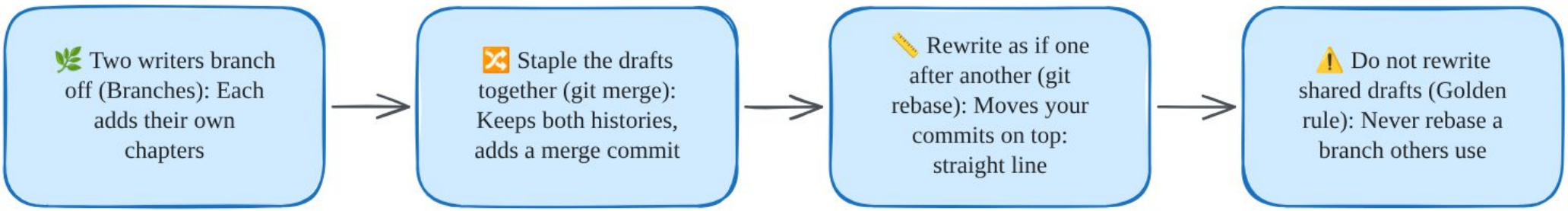
Think of Git like saving a video game



Everyday commands

```
git status                  # what changed?
git add -p                  # stage chunks interactively
git commit -m "Add login form validation"
git switch -c feature/login    # new branch
git pull --rebase           # update without merge bubbles
git push -u origin feature/login
git log --oneline --graph --all
git diff / git diff --staged
```

2. Merge vs rebase

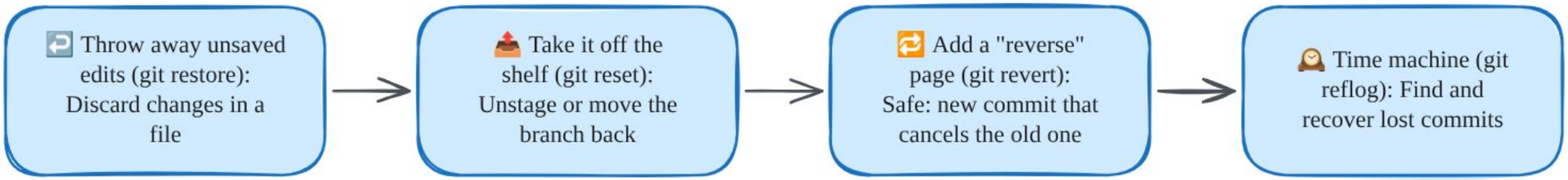


Think of merge vs rebase like combining two story drafts

	merge	rebase
History	keeps the true history + a merge commit	rewrites commits on top of the target → linear
Safe on shared branches?	yes	no — never rebase commits others have pulled
Conflicts	resolved once	possibly per commit

- Squash merge: the whole PR becomes one commit on main (clean history).
- Resolve conflicts: edit the marked files, git add, then git merge --continue / git rebase --continue.
- Golden rule: rebase your own local work; merge shared branches.

3. Undoing things



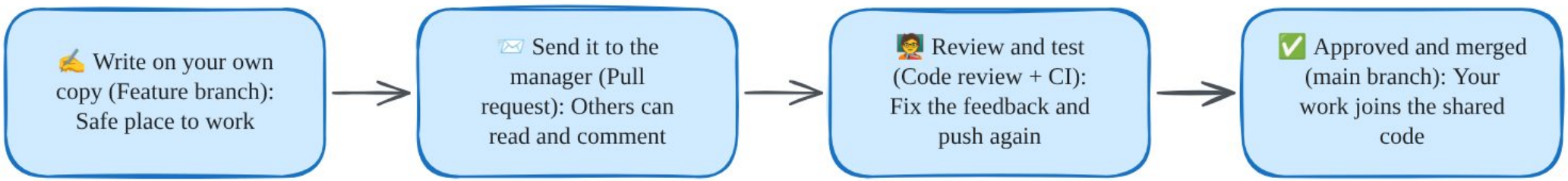
Think of undoing like the undo options in a document

Situation	Command
Discard changes to a file	git restore file
Unstage a file	git restore --staged file
Fix the last commit message / add a file	git commit --amend (only if not pushed)
Undo a pushed commit safely	git revert <sha> (new commit that reverses it)
Move the branch back, keep changes	git reset --soft HEAD~1
Throw local commits away	git reset --hard <sha> (destructive)
Find "lost" commits	git reflog
Park work in progress	git stash / git stash pop
Copy one commit to another branch	git cherry-pick <sha>
Find the commit that broke something	git bisect

Common mistake

git push --force on a shared branch overwrites teammates' work. Use --force-with-lease, and only on your own branches.

4. Team workflow on GitHub



Think of a pull request like submitting a report for review

1. Create a short-lived feature branch from main.
2. Commit small, focused changes with clear messages (Conventional Commits: feat:, fix:, chore:).
3. Open a pull request; CI runs lint, tests and build.
4. Review, address comments, keep the branch updated.
5. Squash-merge, delete the branch, deploy.

- Protect main: required reviews and passing checks.
- .gitignore build output, node_modules, .env.
- Tags (v1.4.0) mark releases; semantic versioning.
- Never commit secrets — if you do, rotate them (history keeps them).

Revise

Commit = snapshot, branch = pointer. Merge shared branches, rebase local ones. revert for pushed commits, reset for local ones, reflog to recover. Small PRs, protected main, CI on every PR.

Rapid-fire Q&A (1/2)

git merge vs git rebase?

Merge joins histories with a merge commit and keeps them as they were; rebase replays your commits on top of another branch for a linear history but rewrites commit ids.

git reset vs git revert?

reset moves the branch pointer (rewrites history, for local work); revert adds a new commit that undoes an earlier one (safe for pushed commits).

git fetch vs git pull?

fetch downloads remote changes without touching your branch; pull = fetch + merge (or rebase).

What is the staging area?

The index where you prepare exactly what goes into the next commit.

How do you recover a commit after a bad reset?

Find its hash in git reflog and reset or cherry-pick it back.

What does git stash do?

Saves uncommitted changes and cleans the working tree so you can switch tasks, then restores them with stash pop.

Rapid-fire Q&A (2/2)

What is cherry-pick?

Applying the changes of one specific commit onto the current branch.

How do you resolve a merge conflict?

Edit the conflicted files to the right content, remove the markers, git add them, then continue the merge or rebase.

Why is force-pushing dangerous?

It overwrites the remote history, deleting commits others may have pulled; use --force-with-lease and only on your own branches.